Software Engineering Assignment #04

AI-assisted software evolution, refactoring and quality improvement

Student	Details
Full name	Zhaulybek Sayat
Student ID	24B031037
Class	Friday 14:00-17:00
Date	8 November 2026
Application	Smart Campus Room Availability Checker

What was delivered

The existing Smart Campus prototype was extended with minimum-capacity and ALL-required-equipment filters. Six fictional rooms, room-specific bookings and the original availability rules are retained. The interface uses a fixed current time of 09:00 and never creates a booking.

Evidence: 33/33 Node tests passed; the original 18 regression tests remain unchanged. Type checking, linting and production build exited successfully. A separate connected-Chrome check passed 18 assertions, including mobile widths of 390 and 320 pixels.

Version references

Published baseline branch: **baseline-a04**
03d7cd408213e0d365785e858171821b331eda14
Published final application branch: **final-code-a04**
55c66603b947e01480ece2c2385ffbe9c66c4610

The **main** branch adds this PDF and documentation. The actual six-room baseline and final code are preserved in separate published branches. All 25 baseline and 51 final-code file hashes match the locally tested snapshots; upload-created commit IDs differ from the original local IDs.

Submission status

The source, tests, logs and this PDF are published without ZIP at:
https://github.com/Sayatikus/Software-Engineering-Assignment-04
Baseline and final-code branches are preserved. SUBMISSION.md contains the exact links and email details. No email has been sent.

1. Codebase analysis and priority plan

Before editing, Codex received the repository map, AC1-AC6, F1-F5 and constraints: React/TypeScript/Vite, fictional fixtures, no login/database/live APIs, fixed clock and immutable booking data. The baseline was saved and its 18 tests were actually run.

Risk and reference	Why it matters	Action
P1 - src/App.tsx: App, handleCheck, TimeField and inline booking table	Form layout, result lifecycle and booking presentation change together. Stored results can become stale after input changes.	Extract TimeForm, RoomFilters and BookingList. Derive the result from current state.
P2 - App parsedStart/parsedEnd/parsedCurrent and timeInputsValid; durationLabel; bookingLogic time helpers	The same time strings are parsed repeatedly in different UI locations. Changes can produce inconsistent feedback.	Remove redundant validation block. Separate time helpers and the selection adapter; retain the original rules API.
P3 - package.json: @supabase/supabase-js	The dependency has no source imports and suggests an unsupported backend. It adds avoidable maintenance work.	Remove the dependency and its lockfile entries. Rerun checks after removal.

Plan recorded before implementation

1. Preserve the working export and establish regression evidence.
2. Add room equipment and pure capacity/equipment filtering.
3. Separate presentation and selection logic; keep AC1-AC6 and refresh results.
4. Run boundary, integration and browser checks; review differences and document limitations.

Rejected AI recommendation

An AI design option was to keep a stored CheckResult and add manual reset/recompute calls to every room, filter and time handler. This was rejected during Codex self-review. The final version stores only whether Check was requested and derives the decision from current inputs. Room switching therefore refreshes the result automatically; filter changes clear hidden selections.

This is a documented AI self-review decision. No separate model conversation or personal student approval is invented. The student should review and explain it before submission.

2. Feature extension and preserved behavior

ID / fictional room	Seats	Equipment	Blocked
r1 Athena Lecture Hall	120	Projector, Whiteboard	No
r2 Newton Lab 201	40	Whiteboard, Computers	No
r3 Curie Seminar Room	20	Projector, Whiteboard	Yes
r4 Da Vinci Auditorium	200	Projector, Microphone	No
r5 Turing Computer Lab	35	Computers, Projector, Whiteboard	No
r6 Plato Discussion Room	16	Whiteboard	Yes

Every room includes ID, name, building, capacity, equipment and blocked status. Separate bookings reference roomId. checkAvailability selects only bookings belonging to the chosen room; no global schedule is applied to every room.

Rule	Implementation and observable result
F1 - minimum capacity	capacity >= attendees. Exactly 40 includes Newton; 41 excludes it. Invalid, zero and fractional attendees show feedback/no matches.
F2 - all equipment	required.every(item => room.equipment.includes(item)). 35 + Projector + Computers leaves only Turing.
F3 - blocked room	Blocked facilities may match filters but the availability decision is always Unavailable. Curie demonstrates this.
F4 - no matches	201 attendees produces an explicit empty state suggesting fewer attendees or less required equipment.
F5 - refresh result	After checking Athena at 10:00-11:00, switching to Turing immediately changes Unavailable to Available. Switching to Curie gives blocked feedback.

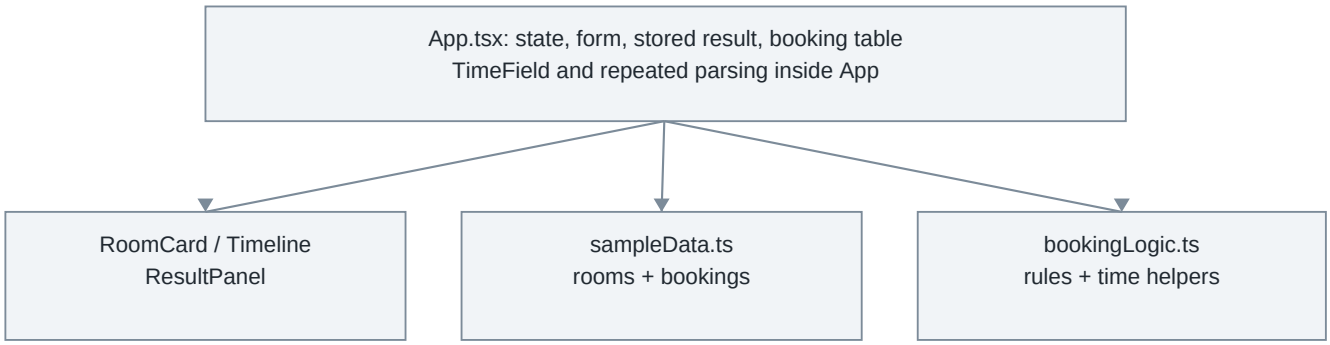
AC1-AC6 remain the contract

AC1: 0 <= start < end <= 1440 and start is after 09:00. AC2: maximum 120 minutes. AC3: blocked rooms rejected. AC4: strict overlap is start < booking.end AND booking.start < end; touching endpoints are allowed. AC5: show a decision without changing bookings. AC6: understandable messages and usable desktop/narrow layouts.

The UI permits 24:00 as an end time. A 24:00 start cannot satisfy start < end within the same day. All data is fictional, and the application is an availability checker rather than a booking confirmation service.

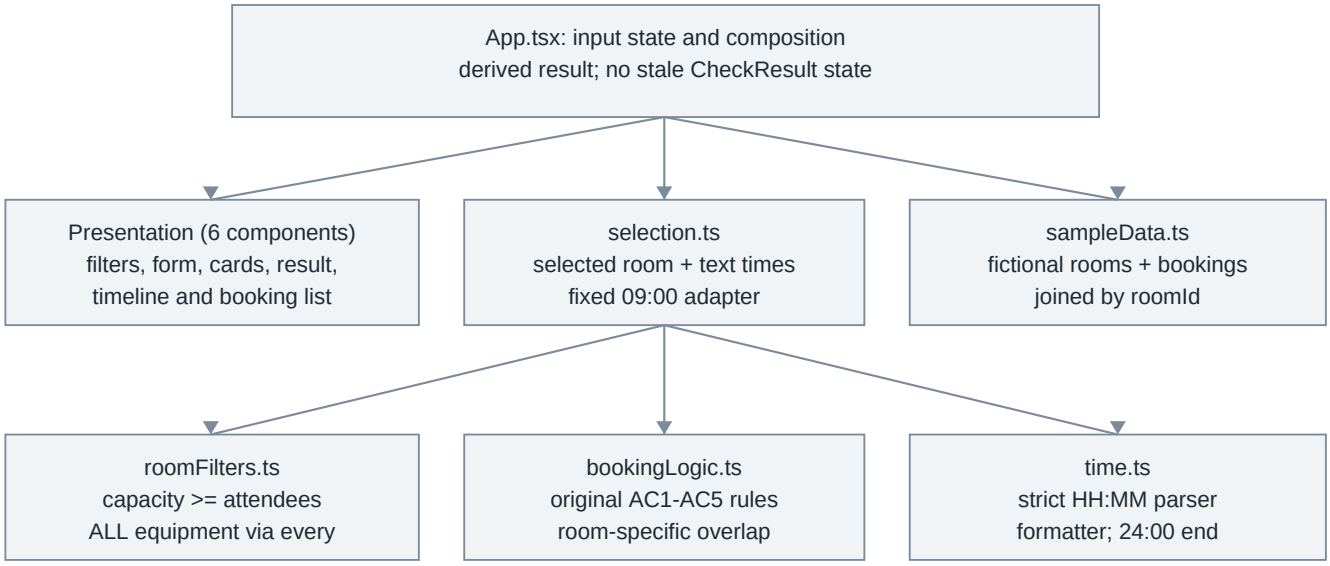
3. Before and after module diagrams

Before - baseline-a04



types.ts supplies Room, Booking and CheckResult contracts

After - final-code-a04



types.ts: shared contracts; bookingLogic re-exports time helpers for compatibility

Decision 1 - pure rules: capacity/equipment filtering is independent of React. Tests assert explicit room IDs without needing a browser, and equipment uses AND semantics.

Decision 2 - derived results: evaluateSelection adapts current input strings and the chosen matching room to the original checker. This eliminates stale saved results and makes F5 directly testable.

Decision 3 - compatibility: checkAvailability keeps its original signature and roomId guard. Existing regression tests run unchanged; bookingLogic re-exports time helpers so retained components continue working. Mermaid and SVG versions of these diagrams are supplied in docs/.

4. Automated tests and actual terminal evidence

Environment: Node 24.13.1, npm 11.8.0, Vite 5.4.8, macOS. Tests use fixed fixtures and 09:00, so outcomes do not depend on the real date. Node >=22.18 is required for direct TypeScript test imports.

Command / evidence file	Actual result
node --test tests/booking.test.mjs (baseline)	18 tests, 18 passed, 0 failed
npm test / final-tests.txt	33 tests, 33 passed, 0 failed
npm run typecheck / final-typecheck.txt	Exit 0
npm run lint / final-lint.txt	Exit 0; no final lint warnings
npm run build / final-build.txt	Exit 0; old Browserslist dataset warning retained
git diff --check / diff-check.txt	Exit 0
Connected Chrome / browser-connector-ui.txt	18/18 real assertions passed; separate from Node total
PLAYWRIGHT_CHANNEL=chrome npm run test:ui	Launch failed before cases: SIGABRT/EPERM. 0 UI cases executed by this command.

Coverage of required boundaries

Requirement / cases	Independent expected result
AC1: T02-T05, T17-T18 and parser tests	Reject now/past, reversed/zero/invalid intervals; allow end 24:00. Reject NaN, Infinity and fractional minute API input.
AC2: T06 / T07	Exactly 120 minutes accepted when free; 121 rejected.
AC3/AC4: T13, T01, T08-T12, T14, T16	Blocked rejected; touching before/after allowed; partial, containing, inside and identical overlap rejected; other-room booking ignored.
AC5: T15 and frozen data test	Both order and contents unchanged; frozen rooms/equipment/bookings survive checks.
F1-F4: catalog tests	40 vs 41 attendees, all selected equipment, combined filters, unknown equipment and empty state.
F5: integration + browser assertions	Same interval returns different decisions for Athena and Turing; hidden/unknown room never available.

The terminal log files contain complete actual output and exit codes. The portable Playwright script contains 12 scenarios but is not claimed to have passed here. Connected Chrome provided the actual UI verification. No code coverage percentage or performance improvement is claimed.

UI evidence - successful filtering and availability

Actual connected-Chrome captures from the running local application, 8 November 2026. These are screenshots, not mockups. Captions describe the exact tested inputs.

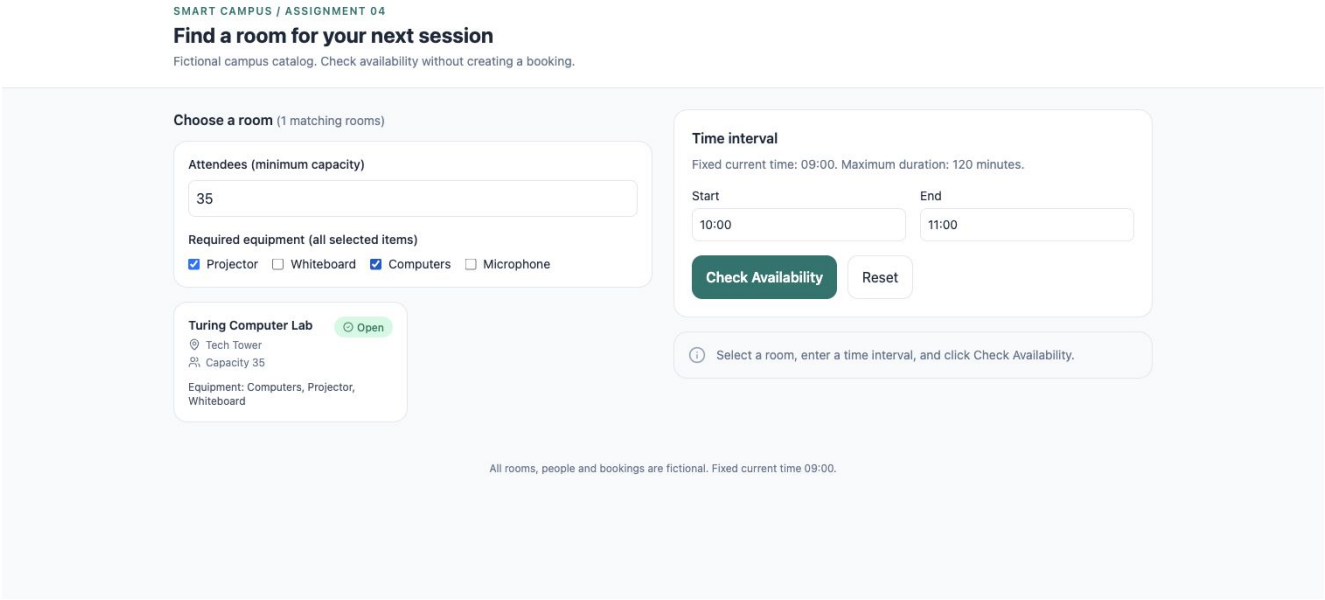


Figure 1. 35 attendees plus Projector and Computers leaves only Turing Computer Lab. The ALL-equipment and exact-capacity rule both apply.

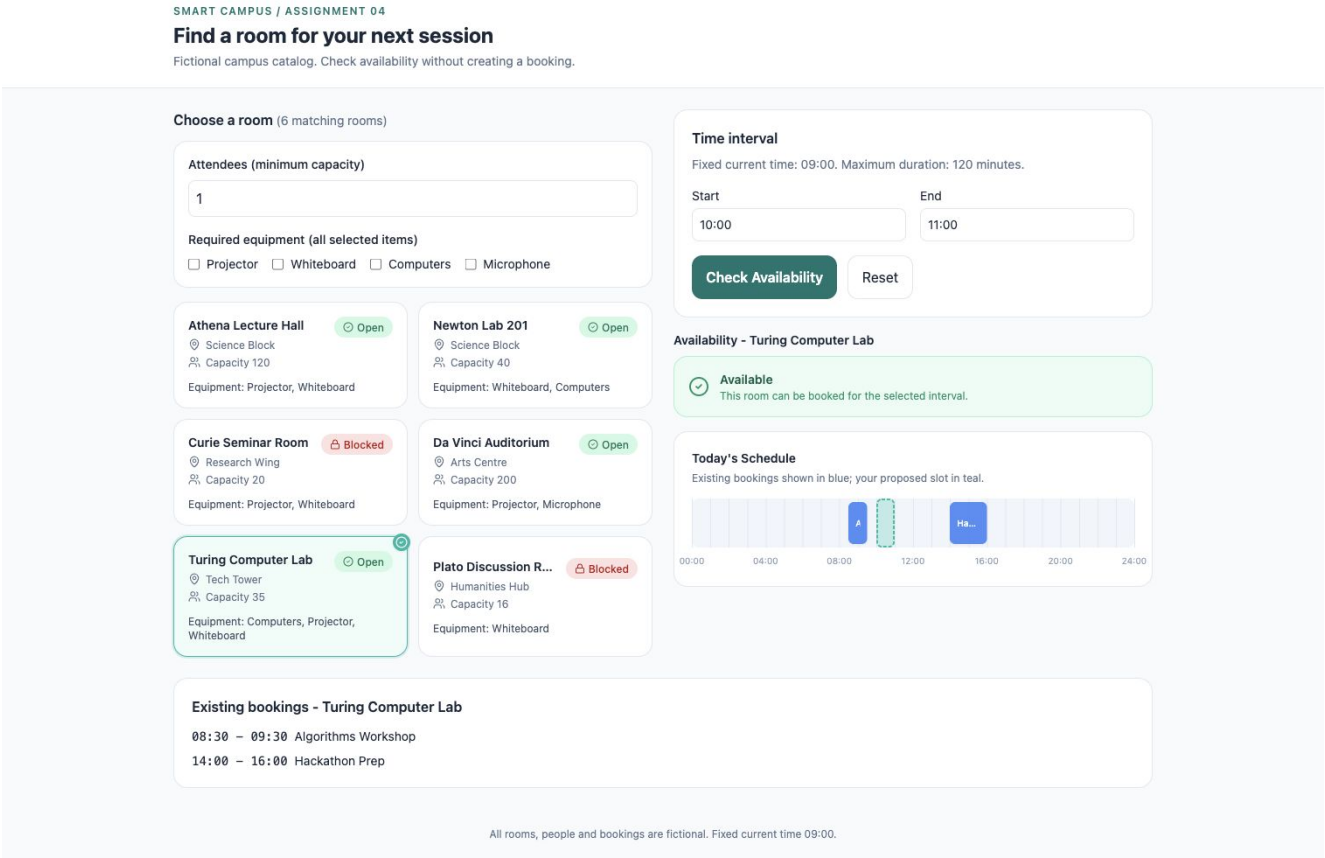


Figure 2. After checking Athena at 10:00-11:00 (overlap), switching to Turing immediately shows Available and its own schedule. No additional Check action is needed for the refresh.

UI evidence - empty result and mobile feedback

F4 was checked with an impossible capacity, and AC6 was checked using real browser viewports rather than a phone-shaped mockup.

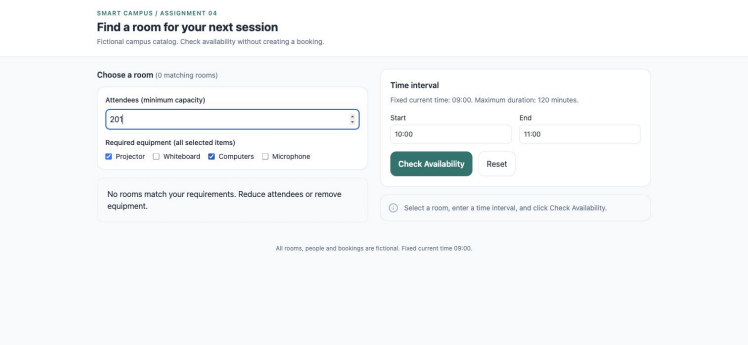


Figure 3. 201 attendees produces zero matching rooms and a clear suggestion to relax requirements.

Mobile verification

At 390px, the form, Available decision, selected-room schedule and booking list remain usable. At 320px, the automated connector assertion also confirmed no horizontal page overflow.

The screenshot on the right is the lower part of the real 390px page, scrolled to the result. The full-height capture is also retained in evidence/screenshots/mobile-390.jpg.

The check is read-only. Repeating it preserves the displayed bookings, while a frozen-fixture Node test verifies data contents and order.

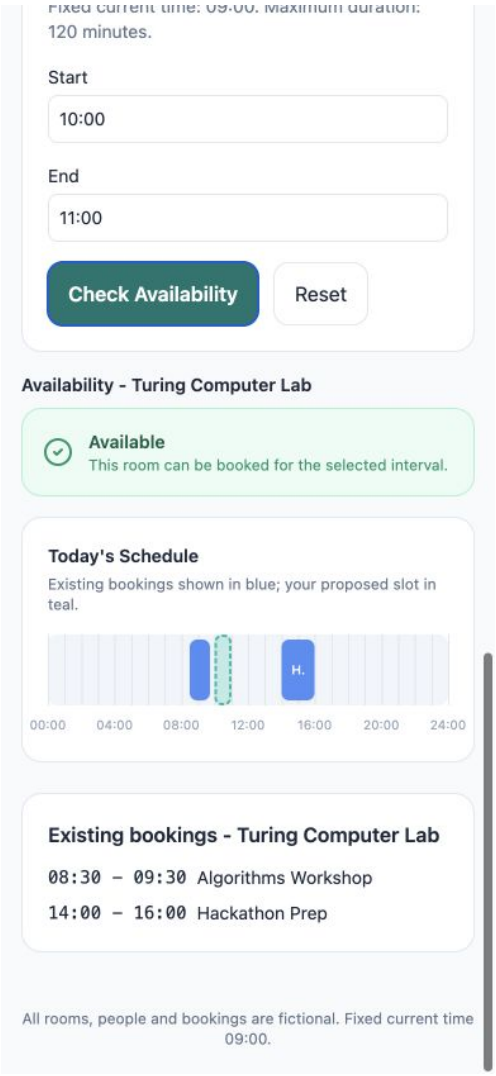


Figure 4. Real 390px viewport.

Blocked room evidence

Curie Seminar Room has no schedule conflicts in the selected interval but is blocked. The connected-browser assertion confirmed an Unavailable result with the blocked reason after switching from Turing. The actual capture is retained as blocked.jpg; the result never carries over as Available.

5. Quality comparison and remaining limitations

Measure	Baseline	Final	How measured
Original regression tests	18/18	18/18	Same unchanged booking.test.mjs on both versions.
All Node tests	18/18	33/33	Actual runner summaries; final adds 15 tests.
Redundant validation blocks	1	0	Manual diff removes timeInputsValid block, which reparsed the same 3 fields. One duplicate location removed.
UI modules (exclude main.tsx)	4	7	App + 3 original / 6 final component files.
src/lib modules	3	6	Count source paths: add time, filtering and selection adapter.
App physical lines	270	141	splitlines count. Formatting affects this; it is not a complexity score.
Rooms and filtering	6, no equipment filters	6, capacity + equipment	Inspect actual catalog and executed F1/F2 tests.

Interpretation

The baseline was already working, so no false failing-baseline claim is needed. The improvement is new tested behavior and clearer responsibility boundaries while all original regressions stay green. More modules alone do not prove better quality; the meaningful evidence is that rules can be tested independently and switching rooms uses current input without stale result state.

The duplication metric is deliberately narrow: one redundant UI validation block removed. It is based on a manual source comparison, not a code-clone detector, maintainability index or percentage reduction across the whole repository.

Limitations kept visible

One fictional day and a fixed clock are intentional. There is no persistence, login or live room API. The browser evidence covers connected Chrome and two narrow viewport widths; it is not a cross-browser or physical-device matrix. The standalone browser launch failed in this environment, even though separate connector assertions succeeded.

No coverage report, complexity benchmark, speed comparison or independent student verification is available. The production build reports outdated Browserslist data; this warning does not change the recorded exit code. Personal review remains an explicit submission step. The published code was verified byte-for-byte against the tested local snapshots.

6. Personal reflection

253 words. Draft for Zhaulybek Sayat to review before submission; no unperformed personal verification is claimed.

For this assignment, I asked Codex to continue my Smart Campus prototype instead of starting an unrelated application. I supplied the assignment PDF, my name and ID, the class time, and the requirement to preserve AC1-AC6. The useful repository context was the existing React interface, booking rules, fictional data, and eighteen regression tests. This made the task more specific than simply asking AI to improve the code.

The most useful design suggestion was to separate filtering from availability. Capacity and equipment decide which rooms are shown, while the original booking rules decide whether the chosen interval is allowed. Another useful decision was to derive the result from the current selection. Keeping a separately stored result would require remembering to reset it in several event handlers, so that option was rejected.

The evidence contains actual automated runs, including the 120/121-minute boundary, touching intervals, exact capacity, and bookings belonging to different rooms. Codex also checked the interface through connected Chrome. I have not yet independently repeated those checks, so I should not describe the automated run as my own manual verification. Before submission, I need to review the room-switching example and explain the overlap condition in my own words.

Tests made the changes easier to judge because the original eighteen checks still pass alongside the new catalog cases. The failed standalone browser launch is also recorded instead of hidden. Next, I would make browser testing easier to reproduce in continuous integration and improve keyboard navigation. I would keep persistence and authentication outside this assignment's scope.

AI-use declaration

OpenAI Codex desktop (GPT-6 family), used 8 November 2026. The exact backend model/version identifier was not exposed. Codex inspected files, implemented changes and executed checks using terminal and connected-browser tools. The existing PDF skill was used to build and verify this report. No new Agent Skill was created for Assignment #04.

All reported tests are tool-executed evidence. The author should only retain a first-person manual-verification claim after personally repeating the relevant action. This draft keeps that boundary explicit.

AI evidence appendix and submission details

Selected input / output	Decision / verification
User request: continue Smart Campus; retain AC1-AC6; implement F1-F5; no ZIP; genuine evidence.	Read assignment and available lecture/practice PDFs. Reuse actual local export; preserve baseline commit and 18-test output.
Pre-edit plan: save baseline, add filters, separate responsibilities, then verify.	Implemented pure filter and selection adapter, extracted form/filter/list components. Unchanged regressions remained green.
AI option: keep stored result and reset/recompute in every event handler.	Rejected during AI self-review. Derived result chosen; F5 integration/browser switching verifies it.
Problem: standalone Playwright launch aborted; first lint had a component-export warning.	Failure retained; actual UI checked through connected Chrome. Unnecessary export removed, final lint rerun successfully.
Generated claims checked against evidence.	33/33 Node results, 18 connector assertions, frozen fixture checks, exact capacity/ALL equipment, real screenshots. No invented human approval.

Sources used directly

Assignment #04 (32 pp.): pp. 9-13 scope/criteria; pp. 14-19 six tasks; pp. 25-29 evidence, submission and deadline.
Lesson #07 Lecture (59 pp.): pp. 8-9 structured context; p. 15 iterative tool loop; p. 18 human review.
Lesson #08 Lecture (68 pp.): pp. 9-14 agent tools and existing skills.
Lesson #08 Practice (35 pp.): p. 8 AC contract; p. 9 debugging workflow; p. 28 real regression output.
Local file named Practice 7 actually identifies itself as Practice #06 (25 pp.): pp. 8-9 Smart Campus AC; p. 21 retest.

Source-access limitation: the separate Lesson #07 Practice PDF mentioned in the earlier chat was not accessible. Its contents are not claimed as directly read. The inspected assignment and Practice #08 provide the implemented test contract. Source file hashes are included in evidence/source-hashes.json.

How to submit without ZIP

The separate repository is published under Sayatikus:
https://github.com/Sayatikus/Software-Engineering-Assignment-04
Use its baseline-a04 and final-code-a04 branches for the initial/final sources; main includes the report. All 25 baseline and 51 final-code file hashes match the tested local versions. Attach this PDF and include the repository and version links in the email.

To: kibo.kz.contact.01@gmail.com
Subject: Software Engineering - Assignment #04 - Zhaulybek Sayat
Deadline: 8 November 2026, 23:59:59 Asia/Almaty. No email has been sent.